Token Importance Unit
Interface Specification

LonghornSilicon LLM Inference Accelerator

Block 3 of 4 — H2O Heavy-Hitter Eviction Core

Version: `tiu-isa-0.1` (first public draft)
Date: 2026-07-18
Status: Pre-tape-out stub for compiler integration
Author: Chaithu Talasila, The University of Texas at Austin
Reference: `LonghornSilicon/token-importance-unit`

Scope. This document describes the externally-visible interface to the `token_importance_unit` block as it will appear on the chip, on an FPGA prototype (Xilinx ZCU102/104), or in the bit-accurate software model. A compiler backend targeting the TIU writes against this interface; the hardware implementation must conform to it. This specification is stable for the Token Importance Unit block only and will be unified with the rest of the LonghornSilicon ISA when the Attention Compute Unit, KV Cache Engine, and Memory Hierarchy Controller blocks land.

Contents

1. Block Overview

The Token Importance Unit is the H2O heavy-hitter eviction core for the KV cache. It maintains N_SLOTS KV-cache slots, each holding an accumulated attention-mass score (the heavy-hitter-oracle statistic). It exposes three operations and one combinational tier read:

$$\begin{aligned}
\texttt{LOAD}(s):&\ \texttt{score}[s] \leftarrow 0,\ \texttt{valid}[s] \leftarrow 1\\
\texttt{ACC}(s,w):&\ \texttt{score}[s] \leftarrow \text{sat_add}(\texttt{score}[s], w)\\
\texttt{EVICT}():&\ \rightarrow \arg\min_{k:\,\texttt{valid}[k]} \texttt{score}[k]\ \text{ (then free it)}\\
\texttt{TIER}(\theta):&\ \texttt{tier_keep}[k] = \texttt{valid}[k] \wedge (\texttt{score}[k] \geq \theta)
\end{aligned}$$

- **ACC** adds the per-step attention mass a cached token just received, one weight per cycle, saturating at $\texttt{SCORE\_MAX} = 2^{\texttt{SCORE\_WIDTH}} - 1$. Accumulators are *distributed* (one saturating adder per slot, no shared broadcast net).
- **LOAD** installs a fresh token: zero its score, mark it valid. LOAD has priority over ACC when both target the same slot in the same cycle.
- **EVICT** runs a *serialized argmin* (one comparator, no wide combinational min tree) over the valid slots, returns the minimum-mass slot — the token to drop — then frees it.
- **TIER** is combinational: N_SLOTS parallel comparators emit a per-slot keep/demote flag against the programmable TIER_THRESHOLD.

The H2O recent-window and KV-budget bookkeeping is a control-layer wrapper, not part of this core. The silicon core is *accumulate + argmin + tier*; the software layer decides *when* to LOAD and EVICT (see `sw/reference_model/example_compiler_use.py`). This mirrors how block 1 shipped just the ratio gate and left the tile loop to the compiler.

Latency: LOAD and ACC complete in one cycle from idle; EVICT takes N_SLOTS + 1 cycles (the serialized argmin scan plus the result). TIER is combinational.
Throughput: one LOAD/ACC per cycle; one eviction per N_SLOTS + 1 cycles.
Footprint: 95 flip-flops at N_SLOTS = 8; 13 833 μm^2 die, $\sim$0.58 µW at SkyWater Sky130A (N_SLOTS = 4 physical proxy; see §6 and `docs/sky130_signoff.md`).
Bit-exact reference: `sw/reference_model/tiu_ref.py` (40/40 RTL replay evictions match).

2. Address Space and Memory Map

The block exposes a 256-byte AXI-Lite slave register window. All registers are 32-bit, word-aligned. The base address is set at chip integration time (e.g., 0x4000_2000 on the ZCU102 PYNQ overlay). $\texttt{SLOT\_WIDTH} = \lceil \log_2 \texttt{N\_SLOTS} \rceil$.

Conventions. RW = read/write; R = read-only; W = write-only trigger; (syn) = value fixed at synthesis time. Writes to read-only registers are silently dropped; reading reserved offsets returns zero. OP_* writes are ignored while STATUS.scan_in_progress is asserted; a compiler polls STATUS.idle before the next op, or uses the streaming interface (§3), which handles backpressure in hardware.

Table 1: AXI-Lite register window (offsets relative to base address).

Offset	Name	Access	Reset	Purpose
0x00	CTRL	RW	0x0	bit[0]: soft_reset (write-1 to pulse; clears valid[]); bit[1]: enable
0x04	STATUS	R	0x1	bit[0]: idle (= !busy); bit[1]: evict_valid; bit[2]: scan_in_progress (= busy)
0x08	INFO_N_SLOTS	R	(syn)	Synthesis-time N_SLOTS
0x0C	INFO_SCORE_WIDTH	R	(syn)	Bits per accumulated-mass score
0x10	INFO_WEIGHT_WIDTH	R	(syn)	Bits per ACC increment
0x14	INFO_VERSION	R	0x0001	bits[15:8] major, bits[7:0] minor
0x18	TIER_THRESHOLD	RW	0x0	Accumulated-mass keep/demote boundary (SCORE_WIDTH bits)
0x1C	OP_LOAD	W	—	Write slot in bits[SLOT_WIDTH−1:0] → LOAD
0x20	OP_ACC	W	—	bits[SLOT_WIDTH−1:0]=slot; bits[16+]=weight → ACC
0x24	OP_EVICT	W	—	Write-1 to bit[0] → trigger a serialized-argmin scan
0x28	EVICT_RESULT	R	0x0	bit[31]: valid; bits[SLOT_WIDTH−1:0]: victim slot
0x2C	TIER_KEEP	R	0x0	bit[k] = tier_keep[k] (1 = keep/CQ-8, 0 = demote/CQ-4)
0x30	SLOT_VALID	R	0x0	bit[k] = valid[k] (diagnostic occupancy)
0x34	SCORE_SEL	RW	0x0	Slot index selecting SCORE_READ (diagnostic)
0x38	SCORE_READ	R	—	Accumulated mass of the SCORE_SEL slot (diagnostic)

3. Streaming Op Interface

For high throughput the op stream is also exposed as an AXI-Stream channel; the AXI-Lite window is then used only for INFO_*, TIER_THRESHOLD, and diagnostics. This is the recommended path — one op per cycle with hardware backpressure, matching the RTL's native op ports.

3.1 s_axis_ops — Operation Input (Slave)

Signal	Width	Direction	Purpose
tdata	2+SLOT_WIDTH+WEIGHT_WIDTH	in	{op[1:0], slot, weight}
tvalid	1	in	Handshake: op is valid
tready	1	out	Handshake: block is ready to accept

op encoding: 0 = ACC, 1 = LOAD, 2 = EVICT (matching the golden trace and analysis/gen_tiu_testvectors.py). For LOAD, weight is ignored; for EVICT, slot and weight are ignored. tready deasserts for N_SLOTS cycles after an EVICT while the argmin scan runs.

Signal	Width	Direction	Purpose
tdata	SLOT_WIDTH	out	The evicted (minimum-mass) victim slot
tvalid	1	out	Handshake: victim is valid (one beat per EVICT)
tready	1	in	Handshake: downstream (KVCE) is ready

3.2 m_axis_evict — Eviction Output (Master)

One beat per EVICT, emitted N_SLOTS + 1 cycles after the EVICT is accepted. tier_keep is not streamed — it is a continuously-valid combinational output the KV cache engine samples when it (re)compresses a token's value (§5).

4. Logical Operations

Operation	Inputs	Outputs	Description
TIU_QUERY	—	INFO struct	Read all INFO_* registers in one transaction
TIU_RESET	—	—	Soft reset: clear valid[] (empty the cache)
TIU_ENABLE	—	—	Set bit[1] of CTRL
TIU_LOAD	slot	—	Install a fresh token (score := 0, valid := 1)
TIU_ACC	slot, weight	—	Add weight of mass to slot (saturating)
TIU_EVICT	—	victim slot	Run argmin; return + free the minimum-mass valid slot
TIU_SET_TIER	threshold	—	Program TIER_THRESHOLD
TIU_READ_TIER	—	keep bitmap	Read TIER_KEEP (per-slot keep/demote)

The block has no other externally-observable side effects: no clock-gating register, no power state machine, and no DFT scan-chain control at this stub level (those land on the chip-level ISA).

4.1 Worked Lowering Example

How a representative compiler IR operation lowers to a sequence of TIU ops. The example uses a generic kv_cache_append op any silicon-agnostic compiler is likely to have; the H2O wrapper is compiler-side.

```
info = TIU_QUERY()                  // -> n_slots=8, score_width=8,
   weight_width=8
TIU_RESET(); TIU_ENABLE()
C = min(round(0.25 * ctx_len), info.n_slots)   // gold 25% KV budget

for t in tokens:
    free = first_invalid_slot()
    if free is None:                // cache full -> evict the argmin
       victim
        victim = TIU_EVICT()        // N_SLOTS+1 cycles, returns slot
        kvce_drop(slot=victim)      // KVCE frees this token's K and V
```

```
        free = victim
    TIU_LOAD(free)                  // install token t
    for (slot, w) in attn_mass_over_cache(t):
        TIU_ACC(slot, w)            // 1 cycle each; accumulate received
           mass
```

Value-tier pass (when the KVCE (re)compresses a surviving token's value):

```
TIU_SET_TIER(threshold)             // e.g. a percentile of accumulated
   mass
keep = TIU_READ_TIER()              // bit[s] = 1 -> value @ CQ-8, else
   CQ-4
for s in occupied_slots:
    kvce_store_value(slot=s, bits = 8 if keep[s] else 4)
```

The same lowering works whether the compiler's target is the Python reference model (phase 0), the FPGA prototype (phase 1), or the silicon chip (phase 3). Only the runtime implementation of the `TIU_*` operations changes underneath — Python call, AXI register write, or PCIe MMIO.

5. Tier Handshake to the KV Cache Engine

The TIU tells the KV Cache Engine (block 2) how to treat each cached token — **keep**, **demote**, or **evict** — via two channels:

1. **Evict** (`m_axis_evict` / `EVICT_RESULT`): the minimum-mass victim's K and V are *dropped* and the slot freed. This is the per-token lever that applies to *both* K and V.
2. **Keep / demote** (`TIER_KEEP` against `TIER_THRESHOLD`): for a surviving token in slot s, the KVCE reads `tier_keep`$[s]$ when it (re)compresses that token's *value*:
   - `tier_keep`$[s]=1$ (heavy hitter) $\rightarrow$ store the **value at CQ-8** (per-token INT8).
   - `tier_keep`$[s]=0$ (demote) $\rightarrow$ store the **value at CQ-4** (per-token INT4).

The tier is a per-token **value**-precision lever only. Keys stay uniform per-channel (per-token key demotion collapses GQA; measured -0.17 in the full stack), so the whole key cache shares one ChannelQuant key tier, set globally. `tier_keep` is emitted as `N_SLOTS` parallel comparators (no read-mux), so it adds no fanout to the argmin datapath — this is what kept the Sky130 sign-off at 0 violations after the port was added. The full protocol, verified end-to-end with the ACU precision controller in the loop, is in `docs/tier_handshake.md`.

6. Synthesis-Time Configuration

The following parameters are baked in at synthesis and cannot be changed at runtime. A compiler reads them via the `INFO_*` registers and tailors its code generation accordingly.

`SCORE_WIDTH` $= 8$ is set from the accumulator-bit-width study (`docs/findings/h2o-deep-analysis.md`): 8 bits is loss-free for the eviction ranking on Qwen2 (Δ -0.002 acc_norm); 6 bits breaks (-0.022). Fewer score bits also shrink the argmin read-mux, which drove the Sky130 max-transition closure.

Parameter	Default	Range (validated)	Notes
N_SLOTS	8	2, 4, 8, 16, 32	KV-cache slots (per head, or per shared group)
SCORE_WIDTH	8	6, 8, 10, 12, 16	Accumulated-mass width; **8 is loss-free**
WEIGHT_WIDTH	8	4, 6, 8	Per-step attention-mass increment width

Physical proxy: N_SLOTS = 4. The shipped RTL default is N_SLOTS = 8 (what the testbench verifies bit-exact). The Sky130 physical run synthesizes an N_SLOTS = 4 proxy via SYNTH_PARAMETERS — exactly as the KV Cache Engine ships a VECTOR_DIM = 8 proxy — to shrink the argmin-mux fanout and clock tree so the clock-root fanout clears the flow's limit; the datapath is parameter-identical. FF count: **95 @ N_SLOTS=8** (real), **53 @ N_SLOTS=4** (proxy). Real N_SLOTS is set per-instantiation. The count tracks the closed form

$$\text{FFs} = \texttt{N\_SLOTS} \cdot (\texttt{SCORE\_WIDTH}+1) + \texttt{SCORE\_WIDTH} + 3 \cdot \texttt{SLOT\_WIDTH} + 3$$

(92 @ N_SLOTS=8; yosys keeps $\sim +3$ slot-index FFs un-merged $\rightarrow$ 95 synthesized, the value the CI gate pins).

7. Bit-Accurate Reference

The Python model at sw/reference_model/tiu_ref.py is the canonical bit-accurate reference for this ISA. A compiler can verify its codegen against the model directly:

```
from tiu_ref import TokenImportanceUnit

tiu = TokenImportanceUnit()
tiu.load(0); tiu.acc(0, 200)
victim = tiu.evict()              # exactly what the chip will return
keep   = tiu.tier(threshold=40)   # per-slot keep/demote, exactly as
    tier_keep
```

The model is verified bit-exact against the RTL by replaying the committed golden trace rtl/tb/testvectors/tiu_trace.hex (48 real Qwen2 tokens, 40 evictions) through both the model and the SystemVerilog replay testbench: all 40 eviction victims agree (sw/reference_model/test_tiu_ref.py). Any divergence between the model and the chip is a bug in this specification, not in the chip.

8. Integration Phases

Throughout all phases, the interface described in this document is the stable contract.

9. Change Log

- tiu-isa-0.1 (2026-07-18): First public draft. Stable for the Token Importance Unit block only. Documents LOAD/ACC/EVICT plus the tier read, the AXI-Lite register and streaming map, the N_SLOTS= 4 physical proxy, and the KVCE tier handshake.

Phase	Timeline	Compiler targets	We provide
0	now	Bit-accurate Python reference model	Model + ISA + parity test
1	when ZCU102/104 arrives	AXI-Lite + AXI-Stream on Zynq UltraScale+	Vivado bitstream + PYNQ driver
2	after all four blocks land	Same AXI interface; tier handshake to KVCE on hardware	Multi-block FPGA project
3	post-tape-out (2027+)	PCIe-attached custom chip	TSMC 16FFC silicon + Linux driver

LonghornSilicon Token Importance Unit — Interface Specification. This document is open and may be redistributed. The TSMC 16FFC PDK referenced in the integration phases is NDA-protected and is not part of this document or the public repository.